

BM20A6100 Advanced Data Analysis and Machine Learning

Final Report

A Methodical Investigation into Deep Learning for Rainfall
Prediction on a Limited Dataset

Climate Analysis A3

Fasie Haider • Haider Ali • Arman Golbidi

Professor: Prof. Lasse Lensu

Teaching Assistant: Dr. Akseli Suutari

LUT University

This report is written in L^AT_EX

All figures are provided in scalable vector format (SVG), so zooming does not reduce image quality.

December 18, 2025

No Artificial Intelligence tools were used in the generation of the code or the written content of this report.

Abstract

The effectiveness of deep learning for rainfall prediction in Sydney, Australia, is thoroughly examined in this stand-alone research. Rainfall prediction remains an inherently challenging task in meteorology due to the chaotic nature of atmospheric processes and the high variability in precipitation patterns. The research progresses from preliminary data discovery and baseline modeling to the deployment and meticulous optimization of sophisticated architectures, such as Transformers and LSTMs. A severely skewed, zero-inflated rainfall distribution was found by preliminary exploratory research. A strong Logistic Regression baseline with an F1-score of 0.588 was produced using 57 engineering features. Our initial deep learning implementations suffered from low performance ($F1 < 0.32$) and training instability. Following a comprehensive diagnostic process, we were able to overcome these issues by boosting the learning rate (from $1e-3$ to $5e-05$) and simplifying the model architecture (from 95K to 11.5K parameters), which improved our best F1-score by more than 30% to 0.384. There was still a significant performance gap from the baseline despite these noteworthy improvements. ROC analysis confirmed this, showing poor AUC values (below 0.61) for all deep learning classifiers compared to a good AUC of 0.821 for the baseline. The insufficient training data (2,284 samples) is the primary barrier for this classification problem, according to our final comprehensive hyperparameter search using the Optuna framework.

Contents

1	Introduction	1
2	Data and Exploratory Analysis	1
2.1	Dataset and Preprocessing	1
2.2	Exploratory Data Analysis (EDA)	1
2.3	Autocorrelation Analysis	3
3	Methodology and Model Formulation	5
3.1	Feature Engineering	5
3.2	Baseline Models	5
3.2.1	Logistic Regression	5
3.2.2	ARIMA Model	5
3.3	Deep Learning Architectures	5
3.3.1	LSTM Model	5
3.3.2	Transformer Model	6
3.4	Training and Optimization Strategy	9
4	Experiments and Results	9
4.1	Diagnosing and Fixing Training Instability	9
4.2	Final Model Comparison	12
4.3	ROC Curve Analysis	13
5	Discussion	14
6	Conclusion	14

1 Introduction

With major ramifications for public safety, agricultural planning, and water resource management, accurate rainfall forecasting is an essential component of contemporary meteorology [1]. In order to estimate rainfall in Sydney, Australia, this study compares cutting-edge deep learning architectures with conventional machine learning techniques in an exhaustive and objective experimental examination. This study intends to investigate the possibilities and limitations of models like LSTMs and Transformers when applied in real-world scenarios with limited data availability, rather than concentrating solely on maximizing forecast accuracy. Every stage of the project is covered in depth in this report, from the first data collection and baseline model building to deployment, diagnostic analysis, and meticulous deep learning model optimization.

2 Data and Exploratory Analysis

2.1 Dataset and Preprocessing

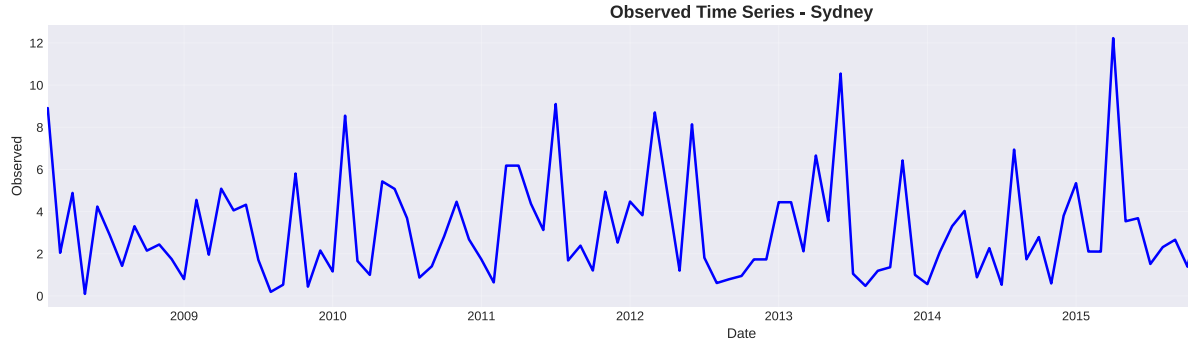
Using the "Rain in Australia" dataset from Kaggle [3], the study focuses on daily observations for Sydney from February 2008 to June 2017. A hybrid imputation technique was used to fill in the missing values in the raw data. The final dataset has **3,271 daily records** after cleaning. dataset we have summarized in Table 1.

Table 1: Dataset Summary for Sydney

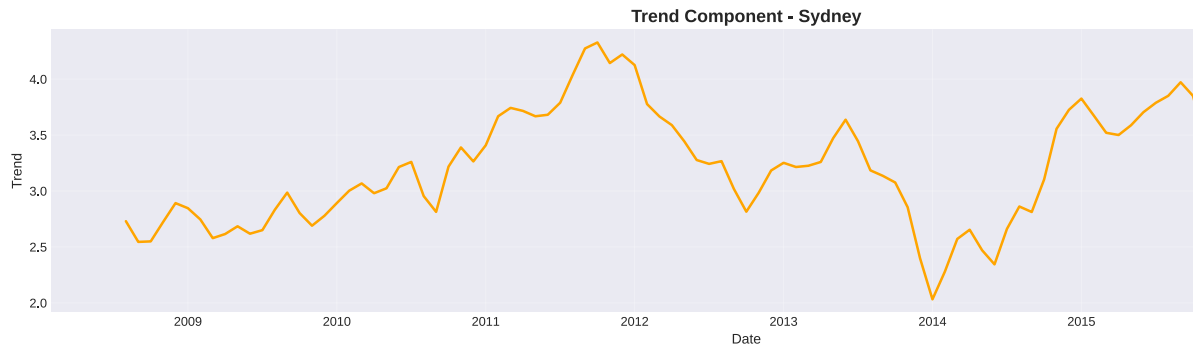
Attribute	Value
Time Period	2008-02-01 to 2017-06-25
Total Observations	3,271
Target Variable	RainTomorrow (Yes/No)
Class Distribution	26% "Yes" / 74% "No"

2.2 Exploratory Data Analysis (EDA)

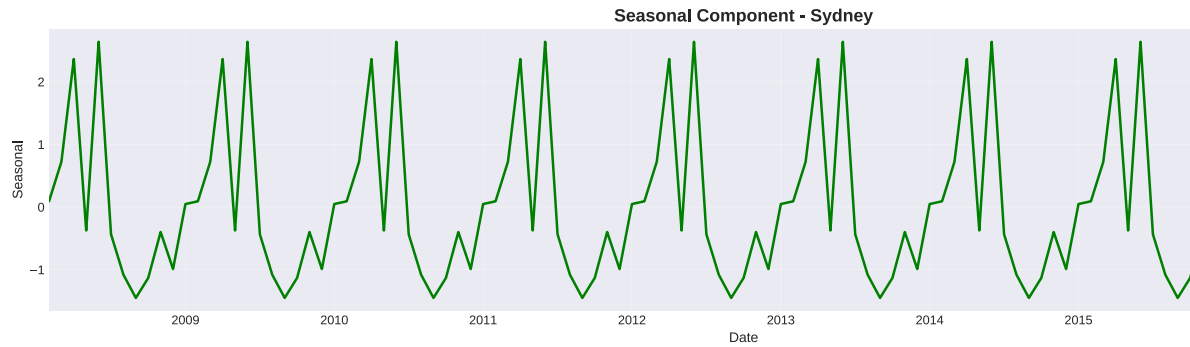
The distribution that EDA showed was right-skewed and zero-inflated (Mean=3.32mm, Median=0.0mm). Although there was no long-term trend (Strength of Trend = 0.0575), time-series decomposition showed a strong seasonal pattern (Figure 1).



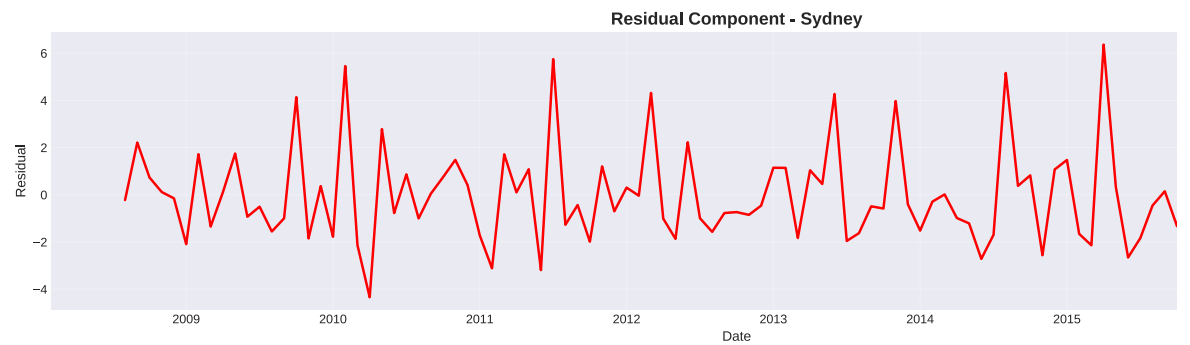
(a) Observed monthly rainfall



(b) Trend component



(c) Seasonal component



(d) Residual component

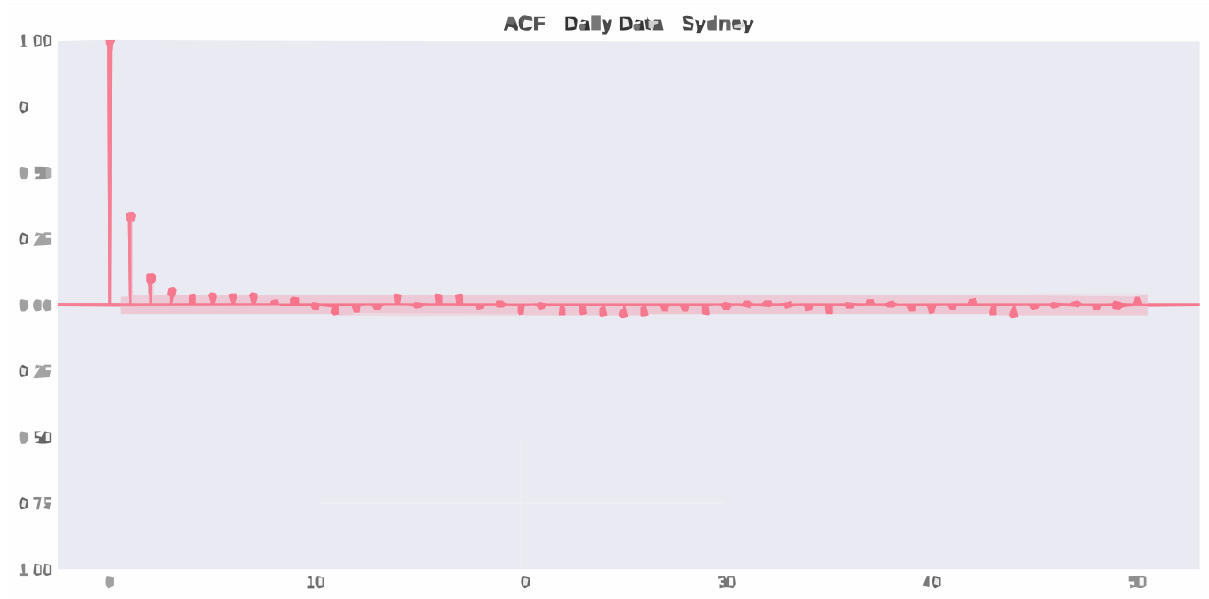
Figure 1: Additive time series decomposition (Sydney).

Observed: the raw monthly rainfall series; Trend: the slowly varying multi-year baseline; Seasonal: the repeating within-year pattern; Residual: the remaining irregular variation after removing trend and seasonality.

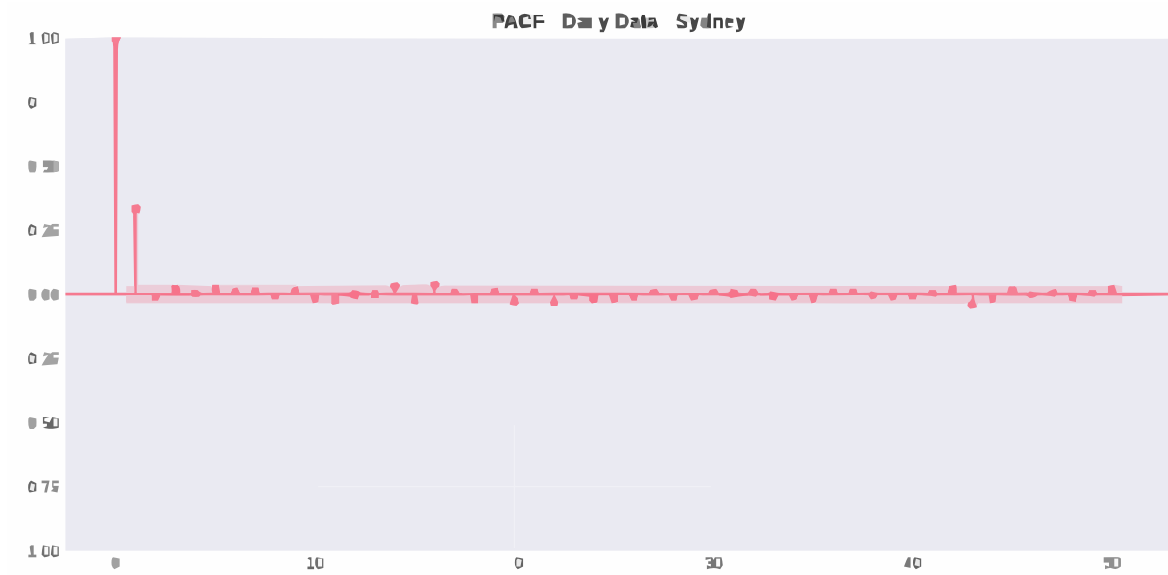
Figure 1 shows that the observed monthly rainfall is highly intermittent, with occasional large spikes (reaching roughly ~ 12 on the plotted scale), consistent with a zero-inflated and right-skewed precipitation process. The trend component changes gradually over multiple years—rising toward 2011–2012, dropping into 2013–2014, and increasing again by 2015—indicating wet/dry regime shifts rather than a simple monotonic long-term trend. The seasonal component is strongly repeatable each year, with pronounced positive peaks and a clear negative trough, confirming that time-of-year contributes a systematic signal. However, the residual still contains sizeable shocks (about ~ -4 to $\sim +6$), implying that a non-trivial fraction of variability is driven by irregular events that cannot be explained by smooth trend/seasonality alone, motivating the use of lag/rolling features and models that can handle stochasticity and extremes.

2.3 Autocorrelation Analysis

A significant peak at lag-1 (correlation = 0.33) in Autocorrelation Function and Partial Autocorrelation Function (PACF) graphs (Figure 2) quickly diminishes. This supports the employment of lag-based features and an AR(1) process for our ARIMA baseline by validating a robust but transient "memory" in the time series.



(a) ACF of Daily Rainfall



(b) PACF of Daily Rainfall

Figure 2: ACF and PACF plots for daily rainfall in Sydney.

ACF shows the correlation between the series and its lagged values (y_t vs. y_{t-k}), while PACF shows the *direct* correlation at lag k after controlling for intermediate lags.

Figure 2 indicates that the rainfall series has a clear **short memory**: there is a noticeable positive dependence at **lag 1** (yesterday vs. today), but the correlations drop quickly and remain close to zero for larger lags. The PACF, in particular, is dominated by the first lag and shows negligible partial correlations afterward, which is consistent with an **AR(1)-type structure**. This supports using an ARIMA(1,0,0) baseline and motivates including lag-based and short rolling-window features (e.g., lag-1, lag-2, and 3–7 day summaries), since most of the usable temporal signal appears concentrated in the most recent few days rather than long-range daily dependencies.

3 Methodology and Model Formulation

3.1 Feature Engineering

To give our models comprehensive, explicit domain knowledge, we developed **57 features**. These include rolling statistics (3, 7, 14-day means/stds), temporal features (cyclical sine/cosine encodings), derived meteorological characteristics ('TempRange', 'PressureChange'), and lag features (historical values at 1, 2, 3, 7-day lags).

3.2 Baseline Models

3.2.1 Logistic Regression

Our primary classification baseline is a logistic regression model. It uses a linear combination of the designed features and the sigmoid function to estimate the probability of rain given the input features, $P(Y = 1 | \mathbf{x})$:

$$P(Y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (1)$$

here $\mathbf{w}$ are learned weights and b is the bias. The model's strong performance is largely attributed to its ability to directly leverage the 57 hand-crafted features.

3.2.2 ARIMA Model

We employ an Autoregressive Integrated Moving Average model for the regression problem. Our ACF/PACF analysis led us to select an ARIMA(1,0,0), or AR(1), process, which is described as follows:

$$y_t = c + \phi_1 y_{t-1} + \varepsilon_t \quad (2)$$

where y_t is the rainfall at time t , c is a constant, ϕ_1 is the autoregressive coefficient for lag-1, and ε_t is white noise.

3.3 Deep Learning Architectures

3.3.1 LSTM Model

Temporal dependencies in sequential data are especially captured by the Long Short-Term Memory (LSTM) network [4]. In contrast to conventional recurrent neural networks, LSTMs use a gating mechanism that regulates input flow between time steps to solve the vanishing gradient issue.

This architecture is especially important for rainfall prediction since weather patterns show temporal correlations: the possibility of precipitation tomorrow is influenced by the atmospheric conditions of today. The LSTM is ideal for capturing both short-term weather variations and longer-term seasonal trends shown in our data because of its capacity to selectively remember or forget information over a range of time frames.

Three gates are used by the gating system. The model may forget unimportant past weather conditions because the *forget gate* $\mathbf{f}_t$ decides which data from the previous cell state should be

disregarded. The model may include current weather observations since the *input gate* $\mathbf{i}_t$ selects which fresh data should be saved. The final prediction and the information flowing to the subsequent time step are controlled by the *output gate* $\mathbf{o}_t$.

Given the input $\mathbf{x}_t$ and the prior hidden state $\mathbf{h}_{t-1}$, we define $\mathbf{z}_t = [\mathbf{h}_{t-1}; \mathbf{x}_t]$. The computations proceed as:

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \end{pmatrix} = \sigma(\mathbf{W}_{gates} \mathbf{z}_t + \mathbf{b}_{gates}), \quad \tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_{cand} \mathbf{z}_t + \mathbf{b}_{cand}) \quad (3)$$

The cell state and hidden output are updated via:

$$\mathbf{C}_t = \mathbf{f}_t \circ \mathbf{C}_{t-1} + \mathbf{i}_t \circ \tilde{\mathbf{C}}_t, \quad \mathbf{h}_t = \mathbf{o}_t \circ \tanh(\mathbf{C}_t) \quad (4)$$

3.3.2 Transformer Model

In contrast to recurrent models, the Transformer architecture [2] uses self-attention mechanisms to analyze whole sequences at once. In addition to providing computational benefits, this parallel processing capability allows the model to explicitly record interactions between any two time steps, regardless of how far apart they are in the sequence.

The self-attention mechanism can directly model how weather conditions from a few days ago might affect present precipitation patterns without the information going via intermediary time steps, which makes it theoretically favorable for rainfall prediction. Complex climatic dependencies, like how a pressure system from a week ago influences the likelihood of rainfall today, might be captured with the aid of this direct access.

The attention weights that each time step should pay attention to each other time step are calculated by the fundamental self-attention mechanism:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (5)$$

where d_k is the key dimension used for scaling, and $\mathbf{Q}$ (queries), $\mathbf{K}$ (keys), and $\mathbf{V}$ (values) are linear projections of the input sequence. By generating a weighted combination of values based on query-key similarity, the softmax operation guarantees that attention weights add up to one.

Since the self-attention operation is permutation-invariant by nature, our implementation (Figure 3) uses learnable positional encodings to give the model sequence order information. Learnable positional encodings are included in our implementation (Figure 3).

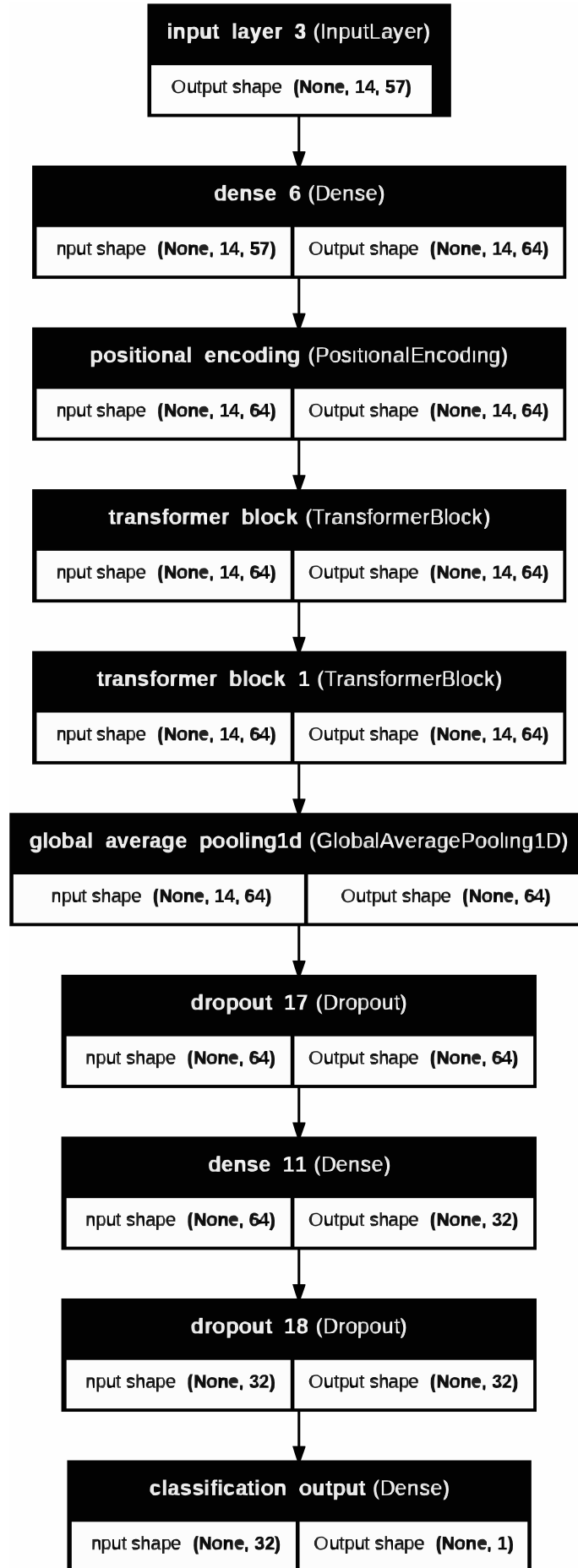


Figure 3: Transformer Classifier architecture

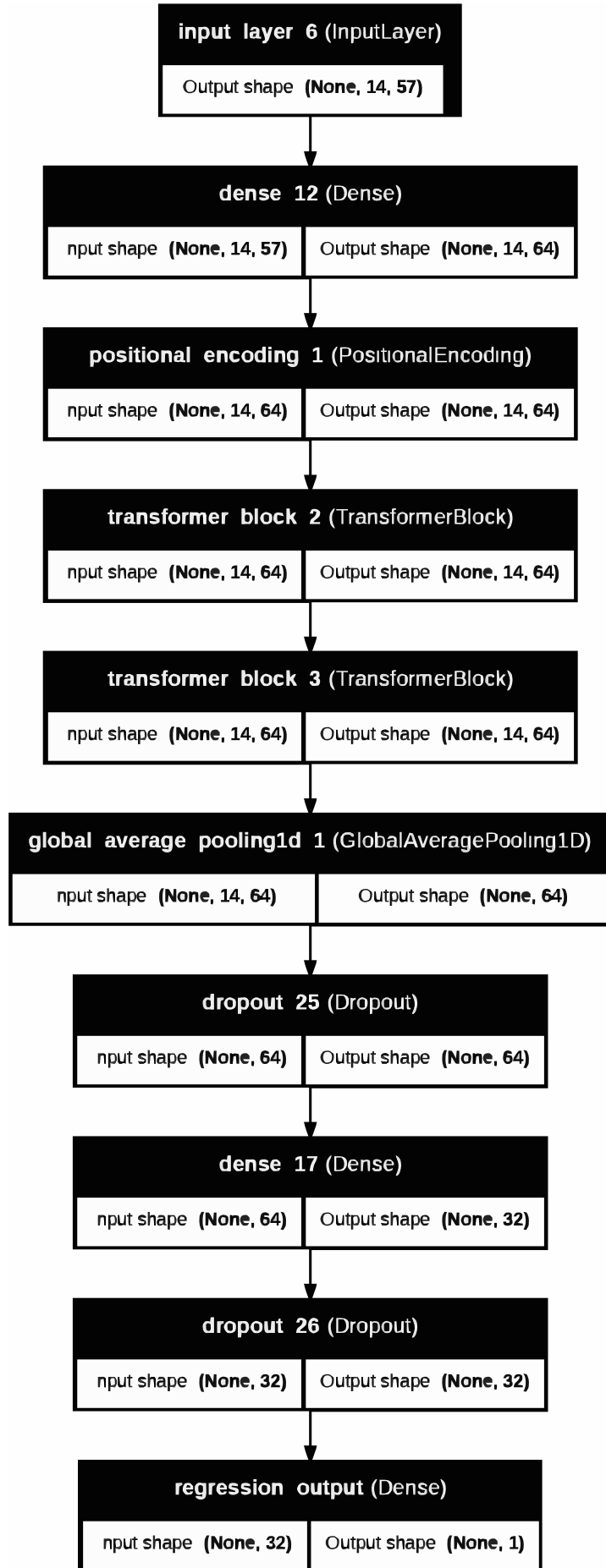


Figure 4: Transformer Regressor architecture

3.4 Training and Optimization Strategy

Initial training with a standard learning rate of 10^{-3} resulted in unstable learning dynamics and inconsistent validation performance. Through systematic experimentation, we identified an optimal learning rate of 5×10^{-5} , which produced substantially more stable convergence and improved generalization. To confirm that these manual adjustments were not incidental, we then performed automated hyperparameter optimization using Optuna [6], conducting a comprehensive search over 50 trials to explore and validate the best-performing training configuration.

4 Experiments and Results

4.1 Diagnosing and Fixing Training Instability

Our initial deep learning models exhibited unstable and unpredictable training losses. Through diagnostic experiments, we confirmed two main causes. First, the learning rate was too high: reducing it to 5×10^{-5} stabilized optimization and improved the F1-score by **+29.1%**. Second, the original architectures were overly complex for the available data; simplifying the design to a **Minimal LSTM** with only 11.5K parameters further improved performance, reaching an F1-score of **0.384** (a **+30.8%** gain over the initial deep model).

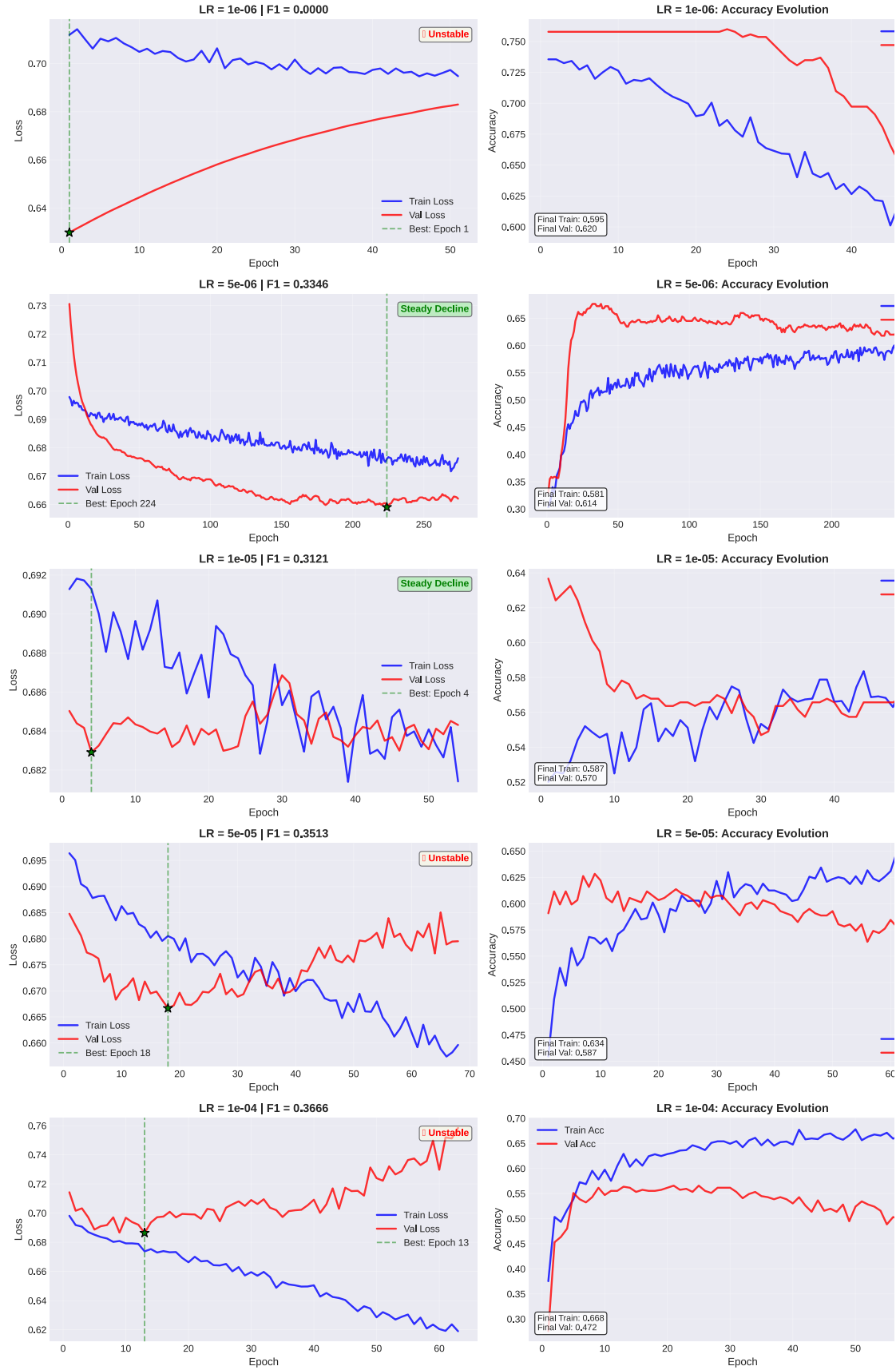


Figure 5: Training curves for different learning rates. Each row corresponds to one learning rate; the left panels show training/validation loss vs. epoch, and the right panels show the corresponding training/validation accuracy evolution.

Figure 5 highlights the typical learning-rate trade-off in small-data training: an overly small learning rate (e.g., 10^{-6}) fails to learn useful decision boundaries (the accuracy does not improve and the reported F1 is effectively zero), while larger rates speed up optimization but can introduce instability and weaker generalization (visible as oscillating loss and a widening gap where training accuracy keeps rising while validation accuracy plateaus or drops, especially at 5×10^{-5} and 10^{-4}). Mid-range values (around 5×10^{-6} to 10^{-5}) show smoother, more consistent convergence with a more stable validation trajectory, indicating better-controlled updates. Overall, the plots suggest selecting a learning rate that yields a steady validation improvement without triggering strong train-validation divergence, since that pattern is a clear signature of overfitting/instability in this limited-data regime.

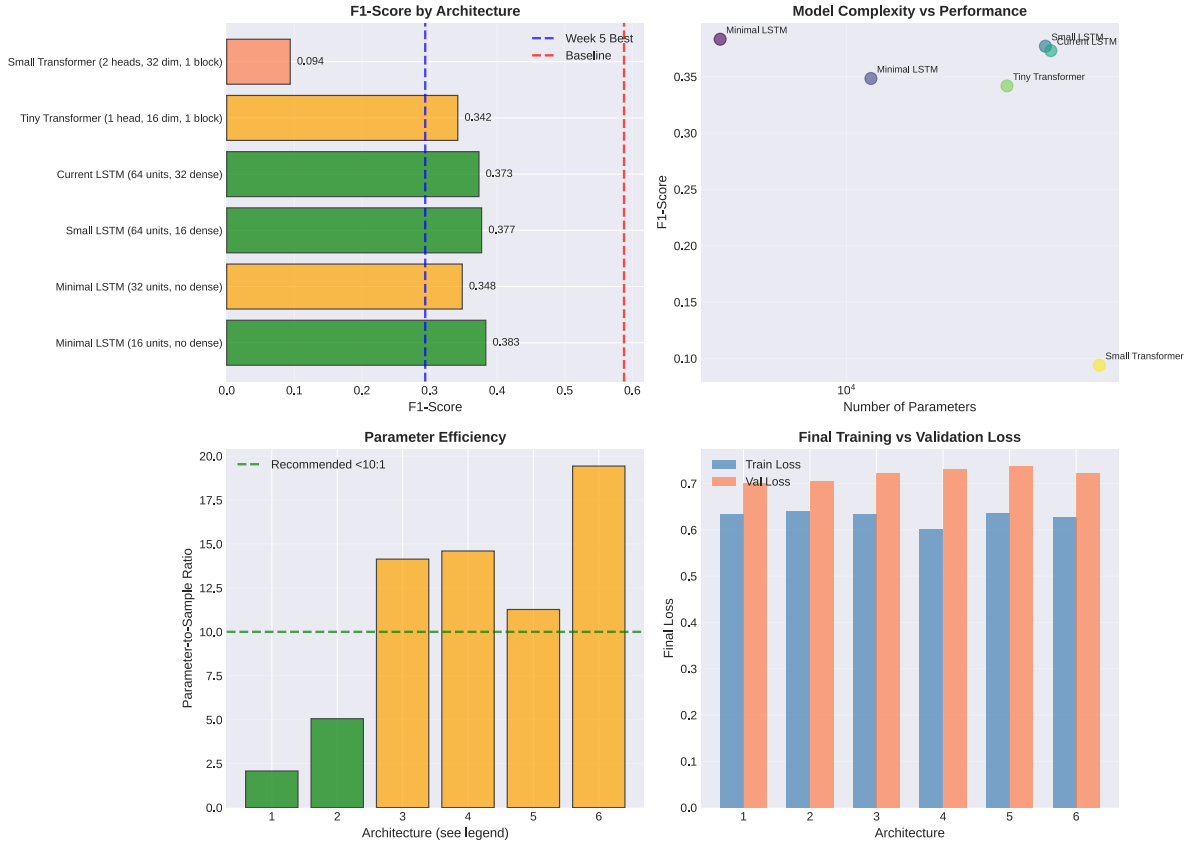


Figure 6: Analysis of architecture.

Top-left: F1-score by architecture with reference lines for the previous best deep model and the logistic-regression baseline; top-right: F1-score vs. number of parameters (complexity) showing the trade-off between size and performance.

Bottom-left: parameter-to-sample ratio (recommended $< 10:1$) as an overfitting/efficiency indicator; bottom-right: final train vs. validation loss highlighting generalization gaps across architectures.

Figure 6 shows that **increasing model complexity does not reliably improve classification performance** in this small-data setting: the best deep-learning result is achieved by a **minimal LSTM** (highest F1 among the tested architectures) while a more complex Transformer variant performs very poorly. The complexity–performance scatter confirms that **compact models can match or outperform larger ones**, implying that extra parameters mostly add variance rather than useful capacity. The parameter-to-sample ratios further support this:

architectures that exceed the recommended $< 10:1$ regime are more prone to overfitting, which is consistent with the final train-validation loss bars where validation loss remains systematically higher than training loss. Overall, the figure suggests that **architectural simplification improves parameter efficiency and stability**, but even the best neural architecture remains well below the strong logistic-regression baseline, reinforcing that feature-engineered classical models are advantaged under limited training data.

4.2 Final Model Comparison

We evaluated our top models against the baselines after a thorough optimization process.

Table 2: Final Performance Summary of All Models

Model	F1-Score	ROC-AUC	RMSE (mm)
Logistic Regression (Baseline)	0.5877	0.821	—
ARIMA (Baseline)	—	—	12.04
Minimal LSTM (32 units, Opt.)	0.3839	0.551	11.99
Multitask Transformer (Opt.)	0.2069	—	11.89

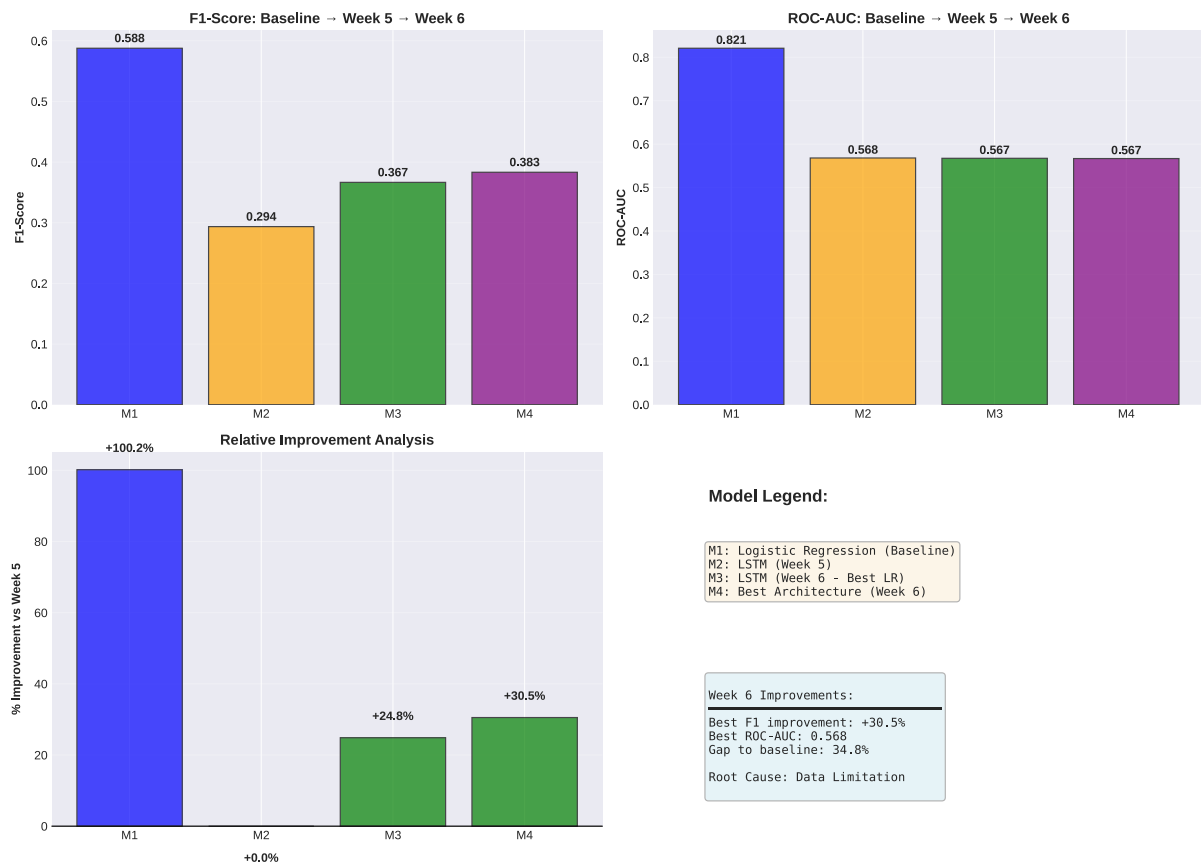


Figure 7: Final comparison of classification performance across development stages.

Top-left: F1-score for the baseline vs. Week 5 and Week 6 deep-learning variants; top-right: ROC-AUC for the same models.

Bottom-left: relative F1 improvement from Week 5 to Week 6; bottom-right: model legend and a short summary of Week 6 gains.

Figure 7 shows that the Logistic Regression baseline (M1) remains clearly dominant ($F1 \approx 0.588$, $\text{ROC-AUC} \approx 0.821$), while the deep-learning models improve over time but still lag behind by a large margin. The Week 5 LSTM (M2) performs poorly ($F1 \approx 0.294$), and Week 6 tuning increases $F1$ to about 0.367 (M3) and 0.383 (M4), which corresponds to roughly a +30% improvement relative to the Week 5 deep model. However, ROC-AUC for the deep models stays almost flat around ~ 0.567 – 0.568 , suggesting that even after optimization the neural classifiers are not learning a substantially better ranking/separation of rainy vs. non-rainy days. Overall, the figure supports the conclusion that the main limitation is the small-data regime: tuning and architectural refinement help stability and $F1$, but they do not close the fundamental performance gap to a strong, feature-engineered classical baseline.

4.3 ROC Curve Analysis

The strongest proof of the performance gap is shown in the ROC analysis (Figure 8). All deep learning models score below 0.61 ("Very Poor"), but the Logistic Regression baseline earns a "Good" AUC of 0.821.

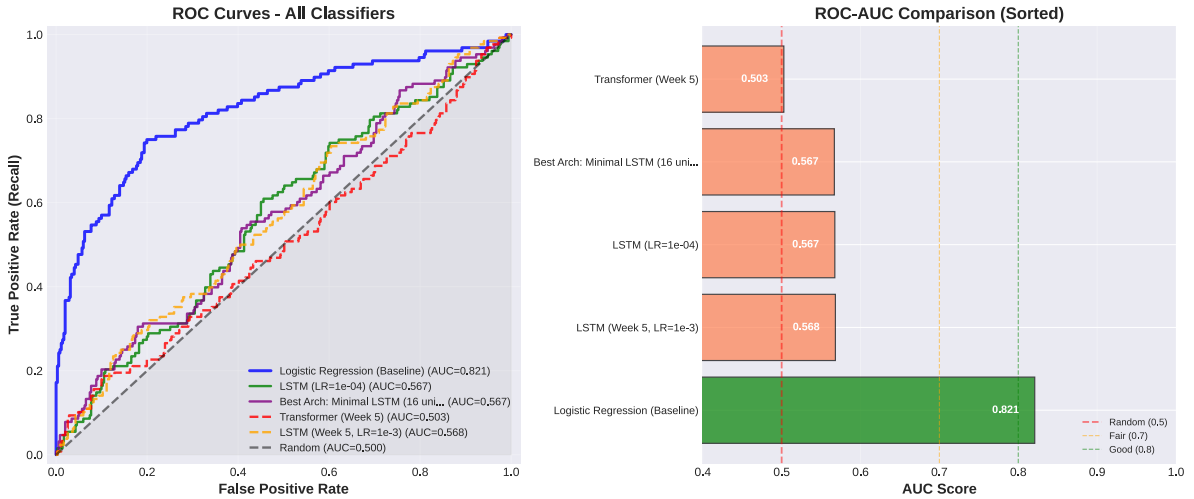


Figure 8: ROC curves for all major classifiers.

Left: ROC curves (TPR vs. FPR) for each model, with the diagonal dashed line representing random guessing ($\text{AUC} = 0.50$).

Right: Sorted ROC-AUC summary, with reference bands indicating Random (0.5), Fair (0.7), and Good (0.8) discrimination.

Figure 8 makes the performance gap unambiguous: the Logistic Regression baseline (blue) stays well above the diagonal over most thresholds and achieves a **Good** AUC of about **0.821**, meaning it can rank rainy vs. non-rainy days substantially better than chance. In contrast, the deep-learning classifiers cluster close to the random baseline: the Transformer (Week 5) is nearly random ($\text{AUC} \approx 0.503$), while the LSTM variants and the minimal-LSTM architecture reach only $\text{AUC} \approx 0.567$ – 0.568 , which is still **Very Poor** discrimination. This implies that, even after tuning, the neural models are not learning a strong separability signal for the binary target; they may fit some patterns but do not generalize into a reliable ranking rule. Overall, the ROC analysis supports the conclusion that in this limited-data regime, feature-engineered

classical modeling provides a much stronger and more stable classifier than the tested end-to-end deep architectures.

5 Discussion

The experimental findings unequivocally show that the amount and kind of the accessible dataset have a fundamental impact on how well deep learning models perform on this classification job.

The main limitation is revealed to be the lack of data. Our top-performing deep learning model ($F1=0.384$) is still 35% below the baseline ($F1=0.588$) even after methodically resolving training instabilities and improving the model architecture. This implies that the intrinsic difficulty of learning from merely 2,284 training sequences is the root of the issue rather than our implementation.

In small data regimes, handcrafted feature engineering outperforms end-to-end learning. The 57 designed features are directly responsible for the outstanding performance of the Logistic Regression model. These features give the algorithm clear, high-quality domain knowledge, whereas a deep learning model would require far more data to construct equivalent representations from start [5].

The Classification vs. Regression Divide. Deep learning did well in regression but poorly in classification, which is an interesting conclusion. Our Multitask Transformer fared better than the ARIMA baseline of 12.04 mm, with an RMSE of 11.89 mm. This suggests that neural networks may find it easier to predict continuous values from sparse input than discrete, unbalanced binary outcomes.

6 Conclusion

This project completed the full machine learning inquiry lifecycle, progressing from initial data exploration and baseline construction to systematic deep learning development, diagnosis, and optimization. By identifying and addressing key training issues, we improved the best deep-learning F1-score by nearly 30%. More importantly, our experiments provided strong evidence that limited data availability is the dominant reason for the weak performance of deep models on this classification task. Along the way, we demonstrated the practical deployment of advanced approaches, including LSTMs, Transformers, and automated hyperparameter optimization using Optuna. Overall, the results support a clear conclusion: for small-scale, tabular time-series classification problems, a well-designed classical method such as logistic regression can remain the most reliable choice, reinforcing the broader lesson that increased model complexity does not necessarily yield better real-world performance.

References

- [1] Wilks, D. S. (2011). *Statistical Methods in the Atmospheric Sciences*, 3rd edn. Academic Press, San Diego, CA.

- [2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. and Polosukhin, I. (2017). Attention is all you need. In: *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008.
- [3] Australian Bureau of Meteorology. (2023). Climate Data Online. Available at: <http://www.bom.gov.au/climate/data/> [Accessed: 10 May 2025].
- [4] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, vol. 9, no. 8, pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [5] Shwartz-Ziv, R. and Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, vol. 81, pp. 84–90. DOI: 10.1016/j.inffus.2021.11.011.
- [6] Akiba, T., Sano, S, Yanase, T., Ohta, T. and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Anchorage, AK, pp. 2623–2631.